Laravel Module Assets — Developer Manual

Overview

This manual documents a practical solution to one of the most common architectural problems in modular Laravel applications: **how to keep JavaScript and CSS files inside their respective modules** while still making them accessible through the web server.

The solution is a single Artisan command — `modules:symlink` — that creates symbolic links from each module's `Assets/` directory into the application's `public/modules/` directory. No build tools, no copying, no configuration overhead.

The Core Problem

Laravel's public directory is the web root. Any file you want to serve to a browser must live under `public/`. In a standard Laravel app, this is fine — you run `npm run build` and Vite places compiled assets in `public/build/`.

In a **modular application**, however, you want each module to be self-contained:

```
Modules/
├── Dashboard/
│   ├── src/Controllers/DashboardController.php
│   ├── src/Views/index.blade.php
│   └── Assets/
│       ├── js/dashboard.js     ← This file needs to be web-accessible
│       └── css/dashboard.css   ← This file needs to be web-accessible
```

The typical workarounds all have significant drawbacks:

Approach	Drawback
Store assets in `public/`	Modules are no longer self-contained
Configure Vite per module	Complex, slow, requires rebuild on every change
Copy assets on deploy	Fragile, easy to forget, creates stale files
Use a package like `nwidart/laravel-modules`	Heavy dependency, opinionated structure

The symlink approach solves all of these problems elegantly.

How Symlinks Solve This

A symbolic link (symlink) is a file system pointer. When you create a symlink at `public/modules/dashboard/js` pointing to `Modules/Dashboard/Assets/js`, the web server sees the files in the `public/` directory — but they physically live inside the module.

```
public/modules/dashboard/js  →  Modules/Dashboard/Assets/js
```

This means:

- **Editing** `Modules/Dashboard/Assets/js/dashboard.js` is instantly reflected in the browser.
- **Deleting** the module directory removes the assets automatically (the symlink becomes broken, which is easy to detect).
- **No build step** is required for plain JS and CSS files.
- **Git** can ignore the `public/modules/` directory entirely.

The Command: `CreateModuleSymlinks`

File Location

```
app/Console/Commands/CreateModuleSymlinks.php
```

Full Source Code

```php
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\File;

class CreateModuleSymlinks extends Command
{
    protected $signature = 'modules:symlink';
    protected $description = 'Create symlinks for module assets to public
directory';

    public function handle()
    {
        $modules = $this->getModuleNames();

        if (empty($modules)) {
            $this->warn('No modules found in the Modules directory');
            return 0;
        }

        $this->info('Creating symlinks for ' . count($modules) . '
module(s)...');

        foreach ($modules as $module) {
            $this->createSymlinkForModule($module);
        }

        $this->info('✓ Module symlinks created successfully!');
        return 0;
    }

    protected function getModuleNames()
    {
        $modulesDirectory = base_path('Modules');

        if (!File::exists($modulesDirectory)) {
            return [];
        }

        return array_filter(
            scandir($modulesDirectory),
```

```
            function ($item) use ($modulesDirectory) {
                return is_dir($modulesDirectory . '/' . $item)
                    && $item !== '.'
                    && $item !== '..';
            }
        );
    }

    protected function createSymlinkForModule($module)
    {
        $sourcePath = base_path("Modules/$module/Assets");
        $destinationPath = public_path("modules/" .
\Illuminate\Support\Str::snake($module));

        if (!File::exists($sourcePath)) {
            $this->warn("  ⚠ Source folder does not exist: $sourcePath");
            return;
        }

        if (!File::exists($destinationPath)) {
            File::makeDirectory($destinationPath, 0755, true);
        }

        $this->createSymlinksForAssets($sourcePath, $destinationPath,
$module);
    }

    protected function createSymlinksForAssets($sourcePath,
$destinationPath, $module)
    {
        $directories = File::directories($sourcePath);

        foreach ($directories as $directory) {
            $dirName = basename($directory);
            $symlinkPath = $destinationPath . '/' . $dirName;

            if (file_exists($symlinkPath) || is_link($symlinkPath)) {
                if (is_link($symlinkPath)) {
                    unlink($symlinkPath);
                } else {
                    $this->error("  ✗ Conflict: $symlinkPath exists as a
real directory.");
                    continue;
                }
            }
```

```
            File::link($directory, $symlinkPath);
            $this->line("  ✓ [$module] $dirName → $symlinkPath");
        }
    }
}
```

How It Works — Step by Step

Step 1: Discover Modules

The `getModuleNames()` method scans the `Modules/` directory at the root of the project and returns the name of every subdirectory. Each subdirectory is treated as a module.

Step 2: Resolve Paths

For each module, two paths are computed:

- **Source**: `base_path("Modules/{ModuleName}/Assets")` — where the assets physically live.
- **Destination**: `public_path("modules/{module_name}")` — where the symlinks will appear. The module name is converted to `snake_case` using `Str::snake()`, so `MyModule` becomes `my_module`.

Step 3: Create Destination Directory

If the destination directory does not exist, it is created with `File::makeDirectory()`.

Step 4: Symlink Each Asset Subdirectory

The command iterates over the subdirectories inside `Assets/` (e.g., `js/`, `css/`, `images/`) and creates one symlink per subdirectory. If a symlink already exists at the target path, it is removed and recreated. If a real directory exists at the target path (a conflict), an error is reported and that directory is skipped.

Module Directory Structure

Each module must follow this structure for the command to work:

```
Modules/
└── {ModuleName}/
    └── Assets/
        ├── js/
        │   └── {module}.js
        ├── css/
        │   └── {module}.css
        └── images/        ← Any subdirectory works
            └── logo.png
```

The `Assets/` directory is the only required convention. The subdirectories within it (e.g., `js`, `css`, `images`) can be named anything — each one becomes a symlink in the public directory.

Installation in an Existing Project

Step 1: Copy the Command

Place `CreateModuleSymlinks.php` in `app/Console/Commands/`.

Step 2: Register the Namespace (if using Modules namespace)

If your modules use a `Modules\` namespace, add the following to `composer.json`:

```
"autoload": {
    "psr-4": {
        "App\\": "app/",
        "Modules\\": "Modules/"
    }
}
```

Then run:

```
composer dump-autoload
```

Step 3: Create Your Module Structure

```
mkdir -p Modules/MyModule/Assets/{js,css}
```

Step 4: Run the Command

```
php artisan modules:symlink
```

Step 5: Reference Assets in Views

```
{{-- Using the asset() helper --}}
<link rel="stylesheet" href="{{ asset('modules/my_module/css/my_module.css')
}}">
<script src="{{ asset('modules/my_module/js/my_module.js') }}"></script>
```

The Asset Helper Class

The project includes an optional helper class at `app/Helpers/ModuleAssetHelper.php` that provides a cleaner API for referencing module assets.

```php
// Get a URL to a specific JS file
ModuleAssetHelper::js('dashboard', 'dashboard.js');
// Returns: http://localhost/modules/dashboard/js/dashboard.js

// Get a URL to a specific CSS file
ModuleAssetHelper::css('analytics', 'analytics.css');
// Returns: http://localhost/modules/analytics/css/analytics.css

// Get all JS files for a module
ModuleAssetHelper::getJsAssets('dashboard');
// Returns: ['http://localhost/modules/dashboard/js/dashboard.js']

// Get all CSS files for a module
ModuleAssetHelper::getCssAssets('settings');
// Returns: ['http://localhost/modules/settings/css/settings.css']
```

In Blade views, use the full class path:

```
<link rel="stylesheet" href="{{
\App\Helpers\ModuleAssetHelper::css('dashboard', 'dashboard.css') }}">
<script src="{{ \App\Helpers\ModuleAssetHelper::js('dashboard', 'dashboard.js')
}}"></script>
```

The Module Service Provider

The `ModuleServiceProvider` at `app/Providers/ModuleServiceProvider.php` automatically discovers and loads module routes and views. Register it in `config/app.php`:

```php
'providers' => ServiceProvider::defaultProviders()->merge([
    App\Providers\AppServiceProvider::class,
    App\Providers\ModuleServiceProvider::class, // Add this line
    // ...
])->toArray(),
```

This provider scans the `Modules/` directory and:

- Loads `Modules/{Name}/src/Routes/web.php` if it exists.
- Loads `Modules/{Name}/src/Views/` as a view namespace (e.g., `dashboard::index`).
- Registers `Modules/{Name}/src/Providers/{Name}ServiceProvider.php` if it exists.

Deployment and CI/CD

What to Commit to Git

Add the following to `.gitignore` to avoid committing generated symlinks:

```
/public/modules/*
```

The `Modules/` directory and all its contents (including `Assets/`) **should** be committed to Git.

Deployment Script

Add the symlink command to your deployment process. For **Laravel Forge** or **Envoyer**, add it to the deployment script:

```
cd /path/to/your/app
php artisan modules:symlink
```

For **GitHub Actions**:

```yaml
- name: Create module symlinks
  run: php artisan modules:symlink
```

Windows Environments

On Windows, creating symlinks requires either:

- Running the terminal as Administrator, or
- Enabling Developer Mode in Windows Settings.

The `File::link()` method in Laravel handles Windows junction points automatically.

Running the Tests

The test suite is located at `tests/Feature/CreateModuleSymlinksTest.php`. It covers 13 scenarios:

Test	Description
`command_creates_symlinks_for_modules`	Verifies symlinks are created for all three demo modules
`symlinks_point_to_correct_directories`	Confirms each symlink resolves to the correct source path
`command_handles_missing_assets_directory`	Ensures the command exits cleanly when a module has no `Assets/` dir
`command_removes_existing_symlinks`	Confirms stale symlinks are replaced on re-run
`asset_files_are_accessible_through_symlinks`	Reads files through the symlink to verify accessibility
`module_asset_helper_returns_correct_paths`	Validates the helper generates correct URLs
`module_asset_helper_retrieves_all_assets`	Confirms the helper lists all files in a module's asset directory
`command_creates_public_modules_directory`	Verifies the `public/modules/` directory is created if absent
`all_modules_are_processed`	Checks that every module in the `Modules/` directory is processed
`command_provides_informative_output`	Validates the command's console output
`module_controller_returns_correct_assets`	Tests the JSON API endpoint for module assets
`module_demo_page_loads`	HTTP test for the main demo page returning 200
`individual_module_pages_load`	HTTP tests for each module's dedicated page

Run all tests:

```
php artisan test
```

Run only the module symlink tests:

```
php artisan test tests/Feature/CreateModuleSymlinksTest.php
```

Frequently Asked Questions

Q: What happens if I add a new JS file to a module's `Assets/js/` directory?

A: Because the symlink points to the entire `js/` directory (not individual files), the new file is immediately accessible through the public URL. No command re-run is needed.

Q: What happens if I add a new subdirectory to `Assets/` (e.g., `Assets/images/`)?

A: You need to re-run `php artisan modules:symlink` to create a symlink for the new subdirectory. Existing symlinks are not affected.

Q: Can I use this with Vite?

A: Yes. You can configure Vite to process files from the `Modules/` directory and output them to `public/build/`. However, for modules that do not require transpilation (plain ES6+ JS and CSS), the symlink approach is simpler and faster.

Q: Does this work with shared hosting?

A: Shared hosting environments often restrict the creation of symlinks. In those cases, you would need to copy the asset files instead of symlinking them. The command can be adapted to use `File::copyDirectory()` instead of `File::link()`.

Q: What is the `snake_case` conversion for?

A: Module names in PHP follow `PascalCase` convention (e.g., `UserProfile`). The `Str::snake()` conversion produces URL-friendly directory names (e.g., `user_profile`), which are then accessible at `/modules/user_profile/js/...`.

Summary

The `CreateModuleSymlinks` command is a lightweight, zero-dependency solution to a real architectural problem. It requires no configuration, no build tools, and no changes to your module code. A single command run during deployment keeps your modular Laravel application's assets organized, accessible, and truly isolated within their respective modules.